Animal Rescue Planner

Team tasks and code design (v2, after the team design discussion). Signatures are language-neutral.

Shared conventions (all members)

Design rules agreed by the team

1. Animals add to cost until they are dropped at a shelter or the overflow site. Rescuing has no meaning beyond delivery.
2. There are no trips. Each vehicle has one event sequence: PICKUP and DROP events, ending with a RETURN to base.
3. Each PICKUP and DROP takes a fixed service time (Config.pickup_time, Config.drop_time). Service time advances the clock and adds no distance.
4. Overflow is a pay-to-use site: its step cost is normal map distance, and each animal sent there adds a flat fee w_over.
5. Vehicles may drive through shelters and the overflow site. Only an explicit DROP event unloads an animal.
6. Vehicles are independent: no road congestion or shared-cell conflicts are modelled.
7. The return to base counts toward distance, but adds no severity delay.
8. Shelters have individual capacities. Capacity is generic (one count per shelter) in the basic version.
9. The shelter count comes from the map template, not from Config.

Phases

- **Phase 1 (basic):** 1 vehicle, generic capacity per shelter, shelters still keep an accepted-species list.
- **Phase 2 (full):** several vehicles, per-species capacity per shelter. Only places marked [P2] change.

Rules that keep Phase 2 cheap

1. Nobody touches shelter counters directly. Use only Shelter.can_accept, remaining, place, release.
2. A solution is a list of routes, one per vehicle, each route a list of Event objects.
3. Costs are computed only through Problem.walk, route_cost, and evaluate. Never add distances by hand elsewhere.
4. Numeric parameters come from Config. No constants inside algorithms.
5. All randomness uses a seeded random.Random instance, so results are reproducible.
6. interfaces.py is frozen. Changes go through a pull request approved by Ali.

Route model

```
PICKUP a1, PICKUP a2, DROP a1, PICKUP a3, DROP a2, DROP a3, RETURN
```

- A DROP event resolves its shelter through the assignment, so an animal can only be dropped where it was assigned.
- Load rules: no PICKUP at full capacity, and no DROP for an animal not on board.
- Every route ends with exactly one RETURN event.
- Service times: each PICKUP adds pickup_time and each DROP adds drop_time to the clock. Only DROP reads the clock for delay.

Cost

```
distance = sum of step costs over all legs, including RETURN
clock    = steps + pickup_time per PICKUP + drop_time per DROP
delay    = sum over DROP events of severity * clock after that drop's service
total    = w_dist * distance + w_sev * delay + w_over * overflow_count
```

Section 1 - Member 1: Foundation and integration (Ali)

Functionalities to implement

1. interfaces.py: ReadMap constants, Event, Config, Map, Animal, Shelter, Vehicle, Problem, Solution, CostBreakdown. Publish first.
2. maps.py: the 5x5, 7x7 and 9x9 templates and MAP_TEMPLATES. Contains no functions.
3. instance.py: generate_instance(config). Places animals on reachable road cells, sets species and severity, creates shelters with individual capacities, and adds the overflow site.
4. main.py: runs the pipeline (instance, distance matrix, assignment, routes, evaluation, display) and prints the result.
5. Publishes the frozen interface and handles change requests from the other members.

Files and code design

```
# interfaces.py
ReadMap = {"Road": 0, "Wall": 1, "Base": 2, "Shelter": 3, "Animal": 4, "Overflow": 5}
PICKUP, DROP, RETURN = "PICKUP", "DROP", "RETURN"

class Event:        __init__(kind, animal_id=None)
class Config:       __init__(map_size, num_vehicles, capacity_mode, w_dist, w_sev, w_over,
                             seed, num_animals, vehicle_capacity, pickup_time, drop_time)
class Map:          __init__(grid); node_count(); neighbors(node) -> [(node, step_cost)]
class Animal:       __init__(id, species, severity, node, needs_shelter=True)
class Shelter:      __init__(id, node, accepted_species, capacity, is_overflow=False)
                    accepts_species(species); can_accept(animal); remaining(animal)
                    place(animal); release(animal); reset(); clone()
class Vehicle:      __init__(id, capacity, base_node)
class Problem:      __init__(map_instance, animals, shelters, vehicles, config)
                    walk(route, assignment, dist) -> (distance, delay)
                    route_cost(route, assignment, dist) -> float
                    evaluate(solution, dist) -> CostBreakdown
                    is_feasible(solution) -> bool
class Solution:     __init__(assignment=None, routes=None); copy()
class CostBreakdown: __init__(distance, severity_delay, overflow_count, total)

# instance.py
generate_instance(config) -> Problem

# main.py
main()
```

Testing

- Interface files import nothing from the other modules, so there are no circular imports.
- Walk trace test: service times advance the clock and affect delay only, never distance.
- generate_instance is reproducible: the same seed and config give the same instance.
- generate_instance raises a clear error for unreachable cells or more animals than road cells.

Section 2 - Member 2: Assignment (CSP)

Functionalities to implement

1. CSP model: each animal that needs a shelter is a variable. Its domain is every shelter that accepts its species, plus the overflow site.
2. Constraints: species acceptance and remaining capacity, both checked only through Shelter.can_accept.
3. Backtracking with minimum-remaining-values ordering and nearest-shelter value ordering (overflow last).
4. Forward checking: when a shelter fills, remove it from other animals' domains. Restore domains on undo.
5. Objective: fewest overflow placements first, then least animal-to-shelter distance. Track the best assignment found.
6. Greedy assignment baseline: animals in severity order, each to the nearest shelter that accepts it.
7. Statistics: nodes explored, backtracks, runtime.

Files and code design

```
# csp.py
solve_assignment(problem) -> dict[animal_id, shelter_id]
greedy_assignment(problem) -> dict[animal_id, shelter_id]

class CspSolver:
    __init__()
    solve(problem) -> dict or None
    select_variable(open_ids) -> animal_id
    order_values(animal) -> list of shelter_ids
    forward_check(animal, shelter) -> bool
    undo(animal, shelter)
    stats                        # nodes, backtracks
```

Testing

- The assignment must agree with Problem.is_feasible on every test instance.
- Hand-made instances: plenty of space, exact fit, and a case that forces overflow.
- Overflow chosen only when no real shelter accepting the species has room.

Section 3 - Member 3: Distance matrix and routing search

Functionalities to implement

1. distance.py: heuristic (Manhattan), a_star with stored paths, points_of_interest, build_distance_matrix.
2. search.py: neighbour_route (swap or reverse two events), greedy_route, optimize_routes.
3. Hill climbing with random restarts and simulated annealing. Both minimise Problem.route_cost.
4. Every candidate route must pass the load rules before it is scored. Infeasible candidates are rejected.
5. greedy_route: pick up the nearest unpicked animal while there is room, drop off when full, return when all are delivered.
6. Statistics: cost evaluations, best cost over time, runtime.
7. [P2] Cross-route moves: transfer an event to another vehicle's route.

Files and code design

```
# distance.py
heuristic(a, b) -> int
a_star(map_instance, start, goal) -> (cost, path)
points_of_interest(problem) -> list of nodes
build_distance_matrix(problem) -> dist[a][b]

# search.py
neighbour_route(route, rng) -> new route
greedy_route(problem, assignment, dist) -> route
optimize_routes(problem, assignment, dist) -> list of routes

class HillClimber:  __init__(seed, iterations); run(problem, route, assignment, dist, restarts)
class Annealer:     __init__(seed, iterations); run(problem, route, assignment, dist, start_temp, cooling)
```

Testing

- A* distances match plain Dijkstra on the same map.
- Every returned route passes is_feasible and never costs more than the starting route.
- The distance matrix is symmetric and has zeros on the diagonal.
- Problem.route_cost is the only cost function used by the search.

Section 4 - Member 4: Visualisation and evaluation

Functionalities to implement

1. experiments.py: solve_exact for tiny instances (enumerates assignments and event orders) and ExperimentRunner.
2. visualize.py: Visualizer draws the map, the routes along stored A* paths, and an animation of the vehicle moving between events.
3. Metrics per run: distance (including return), severity delay, overflow count, total, runtime, and optimality gap on tiny instances.
4. Comparisons: greedy versus CSP assignment, hill climbing versus annealing, severity weight on and off, weight sensitivity, vehicle capacity sweep.
5. CSV output uses len(problem.shelters) - 1 for the shelter count, since Config no longer holds it.
6. [P2] Extra comparisons: one versus several vehicles, generic versus per-species capacity.

Files and code design

```
# experiments.py
solve_exact(problem, dist) -> Solution     # tiny instances only

class ExperimentRunner:
    __init__()
    run(configs, methods, seeds) -> list of result rows
    save_csv(results, path)

# visualize.py
class Visualizer:
    __init__(figsize)
    draw_map(problem) -> axes
    draw_solution(axes, problem, solution)
    animate(problem, solution) -> animation
```

Testing

- On tiny instances, no heuristic may beat solve_exact.
- Start with dummy routes before real solver output is available.
- [P2] Regression check: the full version with one vehicle and generic capacity reproduces the Phase 1 results exactly.